CAREER TRANSITION PLANNING SERVICE

Client Onboarding Form

Design Process & Project Report

Prepared by: Eric Gitonga · July 2026

1. Executive Summary

This document records the complete design process, technical decision-making, and iterative development journey that produced the Career Transition client onboarding form — a professionally branded, dark-themed web application that collects structured intake responses from clients, compiles them into a PDF, and delivers the document automatically to the consultant by email.

The project moved through several platform and technology iterations before arriving at its final architecture: a Python Flask application served by Gunicorn, hosted on Render, with email delivery via the Resend transactional API. The final product requires zero manual intervention from the consultant once deployed — a client completes the form, clicks Submit, and the PDF lands in the consultant's inbox.

Following functional deployment, a comprehensive security audit identified 15 vulnerabilities spanning four severity tiers (4 HIGH, 7 MEDIUM, 4 LOW). All 15 were addressed in a phased hardening programme, elevating the application from an unprotected prototype to a production-grade service with CSRF protection, rate limiting, file type enforcement, a strict Content Security Policy, Subresource Integrity for CDN resources, HTTP security headers, pinned dependencies, and structured audit logging. Security is now a first-class design principle in this codebase, documented in detail in a separate internal audit report (Clients/security.pdf).

A subsequent round of user testing identified that Render's built-in cold-start loading screen — displaying server log output to the client — was confusing and inappropriate for a professional service. A companion Render Static Site was added as the canonical client-facing entry point: a branded loading page with an animated progress bar and estimated countdown that polls the Flask app and redirects automatically once it is ready. User-facing error messaging and startup behaviour were simultaneously formalised in SKILL.md as first-class design principles.

2. Project Brief & Objectives

The brief was to build a structured digital intake form for a career transition coaching practice. Clients completing the form would provide information across nine thematic areas: personal information, motivation and context, target role clarity, constraints and capacity, financial expectations, learning style, network and visibility, past attempts and blockers, and deliverable preferences.

Key requirements:

- Structured, multi-section form with clear visual hierarchy
- Branded PDF output matching the practice's navy / teal / gold colour palette
- Automatic email delivery to the consultant upon submission — no manual steps
- Support for document uploads (CV, LinkedIn export, job description, etc.)
- Publicly hosted on a permanent URL with zero ongoing maintenance burden
- Professional appearance that reflects well on the practice to clients
- No technical knowledge required from the client — the form must be self-contained
- **Security by design** — no credentials exposed in code or templates; all secrets in environment variables; CSRF protection, rate limiting, and file validation enforced at the server level before any business logic runs
- **Production-grade hardening** — Subresource Integrity hashes on all CDN resources, HTTP security headers on every response, pinned dependencies, cryptographically random temp filenames, control-character stripping, and structured submission logging

3. Platform Selection Journey

Arriving at the final hosting stack required evaluating and, in most cases, fully implementing three distinct platforms before settling on the solution that met all requirements. Each iteration produced real, working code and surfaced constraints that informed the next decision.

Phase 1 — Hugging Face Spaces (Gradio)

The initial platform choice was Hugging Face Spaces with a Gradio interface — a well-established combination for rapid Python-based web UIs with free hosting. The form was built using Gradio's component library with the practice's colour scheme applied via custom CSS.

Issues encountered:

- **colorTo parameter rejected** — HF Spaces only accepts a specific set of named colours; 'teal' was not in the list. Fixed by switching to 'indigo'.
- **HfFolder import error** — huggingface_hub ≥ 0.25.0 removed the HfFolder class that Gradio 4.44.0 still depended on. Resolved by pinning huggingface_hub < 0.25.0.
- **CSS / theme API change** — Gradio 4.x moved the css and theme arguments from launch() to Blocks(). Fixed by restructuring the instantiation call.
- **ZeroGPU lock** — the Space was accidentally assigned ZeroGPU hardware; downgrading to CPU-basic requires a Pro subscription. The Space had to be deleted and rebuilt.
- **Policy change** — immediately after rebuilding, Hugging Face announced that Gradio Spaces require a paid plan. Free hosting was no longer available on this platform.

Decision: Abandon Hugging Face Spaces. Seek an alternative hosting provider.

Phase 2 — Google Cloud Run (Aborted)

Google Cloud Run was briefly considered and a Dockerfile and GitHub Actions deployment workflow were created. The client reviewed the setup requirements (GCP project creation, IAM roles, service account key management, GitHub secrets) and concluded the operational overhead was disproportionate for a low-traffic intake form. The implementation was abandoned and all Google-specific artefacts were removed from the repository.

Decision: Abandon Cloud Run. Move to Render, which provides equivalent deployment automation with significantly less configuration.

Phase 3 — Render with Gradio

Render's Python web service was configured with render.yaml, Gradio was retained as the UI framework, and the app deployed. A cascade of compatibility issues followed, each requiring investigation and a targeted fix:

- **Blank page (Gradio 5.x)** — Render's reverse proxy was incompatible with Gradio 5's frontend asset serving. Downgraded to Gradio 4.44.1.
- **Python 3.14 / audioop removal** — Render defaulted to Python 3.14 where the audioop module (a pydub transitive dependency) was removed. Fixed by adding a .python-version file pinning 3.11.9.
- **gradio-client boolean schema bug** — gradio-client 1.3.0 passed boolean values to a function that expected a string, crashing every request. Resolved by monkey-patching json_schema_to_python_type with a try/except guard.
- **Starlette API change** — starlette 1.3.1 broke the TemplateResponse call pattern used by Gradio 4.x. Fixed by pinning fastapi == 0.115.0 to force starlette 0.38.x.
- **Localhost health-check block** — Render's network policy blocks loopback connections; Gradio's post-startup self-ping to 127.0.0.1 always failed. Patched by replacing url_ok with a lambda that always returns True.

Even after all patches, the accumulation of monkey-patches against an ageing framework represented unacceptable technical debt.

Decision: Replace Gradio entirely. Rewrite the application in Flask.

Phase 4 — Flask on Render (Final)

The application was rewritten from scratch in Flask. The form UI was rebuilt in Bootstrap 5 with a full dark theme, all Gradio dependencies were removed, and the PDF generation logic (ReportLab) was retained and refined. The Flask app deployed cleanly on the first attempt. One remaining issue — Render blocking outbound SMTP on port 587 — was resolved by replacing smtplib with the Resend transactional email API, which sends over HTTPS and is never blocked by hosting providers.

Platform comparison summary:

Platform	Outcome	Blocking Issue
Hugging Face Spaces	✗ Abandoned	Paid plan required; ZeroGPU lock
Google Cloud Run	✗ Aborted	Excessive operational complexity
Render + Gradio	✗ Abandoned	Cascade of framework compatibility bugs
Render + Flask	✓ Deployed	None — production-stable

4. Technical Architecture

Backend — Flask + Gunicorn

Flask was chosen for its simplicity, minimal footprint, and zero compatibility issues on Render. Gunicorn serves as the production WSGI server, started with the command specified in render.yaml:

```
gunicorn app:app --bind 0.0.0.0:$PORT
```

The application has two routes: GET / serves the intake form; POST /submit processes the submission, generates the PDF, attempts email delivery, and returns the PDF as a file download. The email outcome (sent / failed / skipped) is communicated back to the browser via an X-Email-Status response header.

PDF Generation — ReportLab Platypus

The PDF is generated programmatically using ReportLab's Platypus layout engine. Each submission produces a multi-page A4 document featuring a navy cover block with the client's name and submission date, nine labelled intake sections, question/answer pairs with bold navy labels and indented answers, and a per-page footer. Multi-select fields are joined as comma-separated strings; blank fields render as an em-dash.

Email Delivery — Resend

Email is sent via the Resend transactional API using the resend Python SDK. Resend operates over HTTPS (port 443) rather than SMTP (port 587), making it immune to port-blocking policies on cloud hosting platforms. The API key is stored as a Render environment variable and is never present in the codebase. All uploaded documents are attached alongside the generated PDF.

Hosting — Render

Render's Python web service provides a permanent public URL, automatic TLS, and GitHub-triggered continuous deployment at no cost on the free tier. render.yaml codifies the service configuration so the hosting setup is reproducible and version-controlled.

Loading Page — Render Static Site

Render's free-tier web service spins down after 15 minutes of inactivity and displays its own 'APPLICATION LOADING' screen — showing server log output — while the instance restarts. This screen is served by Render's reverse proxy before gunicorn binds to the port and cannot be replaced from within application code.

To replace this experience, a companion Render Static Site (career-transition-loading) was added alongside the Flask web service in render.yaml. Static sites on Render have no cold start and serve files instantly. The loading page (loading/index.html) shows a branded dark-theme waiting screen with a spinner, animated progress bar, and estimated 40-second countdown. It polls the Flask app's /_health endpoint every three seconds via the Fetch API and redirects automatically on a 200 response. The canonical client-facing URL is the loading page; the Flask app URL is never shared directly.

The /_health endpoint returns {"status":"ok"} with an Access-Control-Allow-Origin header scoped to the loading site's origin, configurable via the LOADING_SITE_ORIGIN environment variable. This permits the cross-origin Fetch poll without relaxing CORS across the rest of the application.

Security Controls — Flask-WTF, Flask-Limiter, CSP

The application implements all 15 findings from the security audit (detailed in Section 5). The key controls are layered from outermost to innermost:

- **CSRF tokens (Flask-WTF CSRFProtect)** — validated on every POST before any handler logic runs; requests without a valid server-issued token are rejected with HTTP 400

• **Upload size cap (MAX_CONTENT_LENGTH = 10 MB)** — enforced at the Flask framework level; oversized requests return HTTP 413 before reaching route code

• **File extension whitelist (_safe_suffix())** — only .pdf, .doc, .docx, .txt, .jpg, .jpeg, .png are accepted; any other extension returns HTTP 400 before the file is written

• **Rate limiting (Flask-Limiter)** — /submit throttled to 5 requests/minute and 20/hour per IP to prevent Resend quota exhaustion and gunicorn saturation

• **HTTP security headers (@after_request hook)** — every response carries X-Frame-Options: DENY, X-Content-Type-Options: nosniff, a strict Content-Security-Policy, and Referrer-Policy: strict-origin-when-cross-origin

• **Subresource Integrity (SRI)** — Bootstrap CSS and JS loaded from the CDN include SHA-384 integrity hashes; tampered CDN files are blocked by the browser before execution

• **Input length limits (_clip())** — all text inputs truncated server-side before reaching the ReportLab PDF builder, preventing memory exhaustion via crafted payloads

• **Control-character stripping (_sanitize())** — fields used in the email subject and body are sanitised to prevent body-spoofing via injected newlines

• **Random temp filenames (secrets.token_hex(8))** — generated PDFs and uploads use cryptographically unpredictable names, not guessable initials + timestamp

• **Temp file cleanup (finally block)** — all temp files deleted after the response is built regardless of whether an exception occurred

• **Structured submission logging (app.logger)** — every submission is logged with name, email, target domain, upload count, and email outcome for audit and abuse detection

• **Pinned dependencies** — all six packages pinned to known-good versions in requirements.txt; no silent supply-chain updates on each Render deploy

• **Static JS (static/form.js)** — all JavaScript served from a separate file, enabling a strict script-src CSP with no 'unsafe-inline'

---

5. Security Architecture

5.1 Audit Scope and Methodology

A structured code review of all four in-scope files — app.py, templates/index.html, requirements.txt, and render.yaml — was conducted against the OWASP Top 10 and common Flask deployment pitfalls. The review covered injection vectors, authentication and session management, sensitive data exposure, security misconfiguration, cross-site scripting, insecure deserialization, use of components with known vulnerabilities, and insufficient logging and monitoring. No automated scanning tools were used; all findings were identified through manual inspection.

The audit found no critical vulnerabilities (no remote code execution, SQL injection, or hardcoded secrets). All 15 findings address real attack surfaces — from missing CSRF protection enabling forged submissions, to temp files accumulating on disk across every request — that would be exploitable on a public-facing form handling sensitive personal and career information.

5.2 Findings Summary

ID	Finding	Severity	File
S-01	No CSRF protection	HIGH	app.py
S-02	No upload size limit	HIGH	app.py
S-03	No file content validation	HIGH	app.py
S-04	CDN resources without SRI hashes	HIGH	index.html
S-05	No rate limiting	MEDIUM	app.py
S-06	Exception details leaked to client	MEDIUM	app.py
S-07	Temp files never deleted	MEDIUM	app.py
S-08	No HTTP security headers	MEDIUM	app.py
S-09	No SECRET_KEY set	MEDIUM	app.py
S-10	Unpinned dependencies	MEDIUM	requirements.txt
S-11	No input length limits	MEDIUM	app.py
S-12	Control chars in email fields	LOW	app.py
S-13	Predictable temp filename	LOW	app.py
S-14	Extension derived from user input	LOW	app.py
S-15	No submission logging	LOW	app.py

5.3 Remediation Approach

All 15 findings were remediated in three phases ordered by severity. Phase 1 (1–2 hours) addressed all four HIGH findings plus two MEDIUM findings (S-08 security headers and S-09 SECRET_KEY) that were tightly coupled to the Phase 1 infrastructure changes. Phase 2 (2–3 hours) addressed the remaining five MEDIUM findings. Phase 3 (1–2 hours) closed all four LOW findings. Total hardening effort: approximately 6–9 hours, with all 15 findings resolved in a single deployable commit.

As part of Phase 1, all JavaScript was extracted from the inline <script> block in index.html into a separate static/form.js file. This structural change was a prerequisite for enforcing a strict Content-Security-Policy with no 'unsafe-inline' in script-src — a meaningful XSS mitigation that would have been impossible while any inline script

remained in the template.

Note: the full audit report with per-finding code samples, impact assessments, and a phased refactor plan is stored in Clients/security.pdf (gitignored — internal use only).

6. UI / UX Design

The interface was built with Bootstrap 5.3 using the data-bs-theme='dark' attribute for baseline dark-mode component styling, augmented with custom CSS to match the practice's brand identity.

Colour palette:

- **Navy #1B2A4A** — primary background for banners, accordion headers, and the cover block
- **Teal #0E7C7B** — interactive elements: submit button, focus rings, sub-headings
- **Gold #C9A84C** — accent colour for form labels and decorative rules
- **Body #0d1117** — page background (deep dark, matching the GitHub dark theme register)
- **Panel #161b22** — accordion item backgrounds, distinct from page without harshness

Form structure:

The form is organised as a ten-section Bootstrap accordion, with Section 1 (Personal Information) expanded by default. Sections 2 through 10 (Document Uploads) are collapsed and opened on demand, reducing visual overwhelm for what is a substantial intake questionnaire. Field types were chosen to match the nature of each question: radio buttons for mutually exclusive choices, checkboxes for multi-select options, a range slider for hours per week, and textareas for open-ended narrative responses.

Submission flow:

Form submission is handled entirely via the browser's Fetch API — the page never reloads. The submit button is disabled and relabelled 'Processing…' during the server round-trip. On success, the PDF blob is captured from the response, a temporary object URL is created, and the download is triggered programmatically. A persistent download link is also shown on the page. The status banner reports whether email delivery succeeded, failed, or was not configured, based on the X-Email-Status response header.

Form data is submitted as multipart/form-data, including a server-issued CSRF token rendered into the HTML as a hidden field by Flask-WTF's csrf_token() template function. The FormData object captures it automatically alongside all user inputs. All form interaction JavaScript is served from static/form.js rather than inline in the template, enabling a strict Content-Security-Policy that prohibits inline script execution — a meaningful defence against cross-site scripting if an injection point were ever introduced.

7. Key Design Decisions

No client credentials

Removing the email settings section from the client-facing form and moving all credentials to Render environment variables eliminates a significant friction point and prevents clients from encountering technical setup requirements that are irrelevant to their task.

Resend over SMTP

Gmail SMTP on port 587 is blocked by most cloud hosting providers to prevent spam. Resend's HTTPS API bypasses this entirely and requires only a single API key environment variable rather than an email address + App Password pair.

ReportLab over a template engine

Generating the PDF programmatically with ReportLab gives precise control over typography, colour, spacing, and layout. PDF-from-HTML approaches (WeasyPrint, pdfkit) introduce browser rendering dependencies and are heavier to host; ReportLab is a pure-Python library with no system dependencies.

Flask over Gradio / FastAPI

Flask is the thinnest viable option: one file, two routes, no framework magic. It eliminates the compatibility fragility that plagued the Gradio implementation while keeping the codebase easy to maintain and extend.

Bootstrap 5 dark theme

Using data-bs-theme='dark' on the root HTML element gives Bootstrap's component library an appropriate baseline in dark mode, reducing the amount of custom CSS needed while keeping all interactive states (focus, hover, validation) consistent.

X-Email-Status response header

Returning email outcome in a response header rather than the response body allows the PDF to be streamed as the primary response payload while still communicating delivery status to the JavaScript layer — avoiding the complexity of a two-request pattern or a JSON envelope around the binary PDF.

Security as a post-launch priority

Security hardening was conducted as a dedicated phase after functional deployment rather than interleaved with feature development. This sequencing is deliberate — hardening a moving target during active feature iteration introduces the risk of breaking changes in controls not yet under test. A completed, stable feature set was audited once and hardened comprehensively in a single commit.

Static JS file over inline script

Extracting the form's JavaScript from an inline <script> block into static/form.js enables a Content-Security-Policy with no 'unsafe-inline' in script-src. If an attacker finds an HTML injection point, they cannot execute injected scripts because the CSP blocks all inline execution. Style-src retains 'unsafe-inline' because Bootstrap uses inline style attributes for accordion height animations — styles are far less dangerous than scripts.

In-memory rate limiting for single-instance deployment

Flask-Limiter defaults to in-memory storage for tracking request counts, which is appropriate for a single-worker Render free-tier deployment. Request counts are per-process and reset on restart — acceptable given the low traffic profile of a private coaching intake form. A Redis backend would be required if the application were scaled to multiple workers, and is noted as a future upgrade path.

Companion static loading site for cold-start UX

Render's free-tier instances display a server-log loading screen during cold starts — output that is confusing and inappropriate for a client-facing professional service. Because this screen is injected by Render's reverse proxy before any application code runs, it cannot be replaced through Flask routing or templating. A separate Render Static Site — which has no cold start — serves as the canonical entry point. It shows a clean branded waiting screen, polls /_health to detect readiness, and redirects automatically. The pattern completely hides Render's infrastructure from clients while staying within the free tier.

User-facing behaviour as a documented design rule

Tester feedback formalised two principles now codified in SKILL.md: errors shown in the browser must always be plain English — no Python tracebacks, status codes, or server log lines — and the loading page URL is the only URL shared with clients. Documenting these as explicit rules, not just implementation choices, ensures any future changes to error handling or hosting configuration preserve the same standard.

---

8. Development Phases & Effort Breakdown

The following breakdown reflects the actual work performed across the full lifecycle of the project, including all platform iterations and their associated debugging, plus the security audit and hardening programme. Hours are estimates based on typical task durations for work of this nature and complexity.

Phase	Description	Est. Hours
Discovery & Scoping	Requirements capture, field design, section planning	2
Form Architecture	10-section field map, input type selection, validation rules	3
HF Spaces Setup	Initial Gradio form build, README configuration, first deploy	3
HF Spaces Debugging	colorTo, HfFolder, ZeroGPU, css/theme API fixes	3
Cloud Run Spike	Dockerfile, GitHub Actions workflow (aborted)	2
Render / Gradio Setup	render.yaml, Python pin, initial Gradio deploy	2
Render / Gradio Debugging	Blank page, audioop, boolean schema, starlette, localhost	5
Flask Rewrite	app.py routes, form handling, Jinja2 template	4
ReportLab PDF Engine	Cover block, 9 sections, Q&A; formatting, footer	3
Email Integration	SMTP attempt, timeout fix, migration to Resend API	3
UI / Dark Theme	Bootstrap 5 dark mode, custom CSS, accordion, JS fetch	4
UI Refinement	Section removal, heading copy, multi-file upload, headshot removal	2
Deployment & DevOps	render.yaml, env vars, gunicorn config, .python-version	2
Documentation	hosting.pdf, README, docstrings across app.py and generators	4
Testing & QA	End-to-end submission, email delivery, PDF review, edge cases	3
Security Audit	Manual code review of all 4 in-scope files; 15 findings documented in security.pdf	4
Phase 1 Hardening	CSRF (Flask-WTF), upload cap, file whitelist, SRI hashes, security headers, SECRET_KEY; JS extracted to static/form.js	2
Phase 2 Hardening	Rate limiting (Flask-Limiter), exception handling, temp cleanup, pinned deps, input limits	3
Phase 3 Hardening	Control-char stripping (_sanitize), random temp filenames, structured submission logging	1
Security Testing	End-to-end verification: CSRF rejection, rate-limit response, file type rejection, CSP and header inspection	2
Round 2 Feedback — Cold-Start UX	Branded static loading page (loading/index.html), /_health endpoint with CORS headers, Render Static Site service in render.yaml, SKILL.md user-facing behaviour rules	3

Total estimated effort: 60 hours (range: 54 – 66 hrs), comprising 45 hours of product development, 12 hours of security audit and hardening, and 3 hours of Round 2 UX feedback implementation.

9. Cost Analysis

The following analysis presents what a professional consultant or freelance developer would charge for this project in two market contexts. The security audit and hardening programme (12 hours, ~21% of total effort) is listed as a separate line item to make the cost of proper security practice visible and non-negotiable. All KES figures use an exchange rate of KES 130 per USD.

9.1 Nairobi Market

The project spans three disciplines: full-stack web development, UI/UX design, and DevOps / cloud deployment. The blended rate below reflects a mid-to-senior consultant capable of delivering all three, plus a security specialist rate for the audit and hardening work.

Discipline	Nairobi Rate (KES/hr)	Hours	Subtotal (KES)
Full-stack development (Flask, PDF, Email)	2,000 – 2,500	28	56,000 – 70,000
UI / UX design (Bootstrap dark theme)	1,500 – 2,000	6	9,000 – 12,000
DevOps / deployment (Render, env vars)	1,800 – 2,500	5	9,000 – 12,500
Security audit & hardening	2,000 – 2,500	12	24,000 – 30,000
Documentation & QA	1,500 – 2,000	6	9,000 – 12,000

Nairobi market total: KES 107,000 – 136,500 (~USD 823 – 1,050)

*For reference, a fixed-price project quote in the Nairobi market for a deliverable of this scope, polish, and security posture would typically range from **KES 110,000 – 150,000**, including one round of post-launch revisions.*

9.2 Global Market (US / Western Europe)

On international freelance platforms (Upwork, Toptal, Arc.dev), senior Python developers with full-stack, DevOps, and application security capability typically command the following rates in 2026:

Discipline	Global Rate (USD/hr)	Hours	Subtotal (USD)
Full-stack development (Flask, PDF, Email)	85 – 120	28	2,380 – 3,360
UI / UX design (Bootstrap dark theme)	60 – 90	6	360 – 540
DevOps / deployment (Render, env vars)	80 – 110	5	400 – 550
Security audit & hardening	90 – 130	12	1,080 – 1,560
Documentation & QA	65 – 90	6	390 – 540

Global market total: USD 4,610 – 6,550 (~KES 599,000 – 851,000)

*A US-based digital agency or boutique software consultancy billing at agency rates ($150 – $200/hr) would price this project at **USD 8,550 – 11,400**.*

9.3 Cost Summary

Market	Rate Basis	Low Estimate	High Estimate
Nairobi — local clients	KES 1,500–2,500/hr	KES 107,000 (~USD 823)	KES 136,500 (~USD 1,050)
Nairobi — intl. clients	USD 25–45/hr	USD 1,425 (~KES 185,000)	USD 2,565 (~KES 334,000)
Global freelance market	USD 85–130/hr	USD 4,610 (~KES 599,000)	USD 6,550 (~KES 851,000)
US / EU agency rate	USD 150–200/hr	USD 8,550 (~KES 1,112,000)	USD 11,400 (~KES 1,482,000)

Rates based on 2025–2026 market data from Glassdoor, PayScale, Arc.dev, and lemon.io. Security audit and hardening (12 hrs) accounts for approximately 21% of total project effort, reflecting the non-negotiable cost of building a public-facing form that handles sensitive personal and career information. Ongoing hosting costs are zero on Render's free tier; Resend's free tier covers 3,000 emails/month.

10. Final Deliverable

The completed system consists of the following components, all version-controlled in the GitHub repository **ericgitonga/career-transition-intake**:

- **app.py** — Flask application: two routes, PDF generation, Resend email dispatch, file upload handling, and all 15 security controls implemented (_safe_suffix, _clip, _sanitize, CSRFProtect, Limiter, _security_headers, random temp filenames, finally-block cleanup, structured logging). Fully documented with detailed docstrings.
- **templates/index.html** — Dark-themed Bootstrap 5.3 form with SHA-384 SRI hashes on both CDN resources, a CSRF token hidden field, 10 accordion sections, 4 dedicated upload fields plus a multi-select additional documents field. No inline scripts.
- **static/form.js** — Form interaction JavaScript: hours slider, AJAX fetch submission, PDF blob download trigger, email status feedback. Served as a static file to satisfy the strict script-src CSP with no 'unsafe-inline'.
- **requirements.txt** — Six pinned dependencies: flask==3.1.3, flask-wtf==1.3.0, flask-limiter==4.1.1, gunicorn==26.0.0, reportlab==4.4.10, resend==2.32.2.
- **render.yaml** — Infrastructure-as-code: two services — the Flask web service (Python runtime, gunicorn start command, RESEND_API_KEY / SECRET_KEY / LOADING_SITE_ORIGIN env vars) and the companion Render Static Site (career-transition-loading, staticPublishPath: loading).
- **loading/index.html** — Branded dark-theme loading page served by the Render Static Site. Displays a spinner, animated progress bar, and estimated 40-second countdown. Polls the Flask app's /_health endpoint every 3 seconds and redirects on success. This is the canonical client-facing URL — it replaces Render's server-log cold-start screen entirely.
- **.python-version** — Pins Python 3.11.9 to ensure consistent runtime across all Render deploys.
- **SKILL.md** — Consultant workflow guide including a Security First section with a controls reference table, client data handling rules, user-facing behaviour rules (plain-English errors, loading page as canonical entry point), change-management guidelines, and an expanded quality checklist with security checks as the first gate.
- **hosting.pdf** — Illustrated setup guide for deploying and operating the service, covering Render account setup, Resend API key creation, and ongoing maintenance commands.
- **generate_hosting_pdf.py** — ReportLab script that regenerates hosting.pdf when deployment instructions change. Fully documented with comprehensive docstrings.
- **generate_design_pdf.py** — ReportLab script that produces this document. Tracked in the repository; regenerate whenever the design narrative, effort breakdown, or cost figures are updated.

Live URLs:

```
Client entry point (share this): https://career-transition-loading.onrender.com
Flask application (internal): https://career-transition-intake.onrender.com
```

The service is production-ready and hardened. Clients are directed to the loading page URL, which handles cold-start waiting gracefully and redirects automatically to the form. On submission, their completed intake PDF is delivered to the consultant's inbox within seconds, with no manual intervention required, and the submission is logged server-side for audit and abuse detection.